

Session 11

Task1: Implement command history functionality in a shell. The program should store commands, maintain a history buffer, handle capacity limits, retrieve entries, display history, and validate consistency.

Code:

```

rohita@LaxmiDurga: ~
GNU nano 8.7.1

    strcpy(history[MAX_HISTORY - 1], command);
}
}

void show_history()
{
    printf("\nCommand History\n");
    printf("-----\n");

    if (history_count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    for (int i = 0; i < history_count; i++)
    {
        printf("%d %s\n", i + 1, history[i]);
    }
}

int main()
{
    char command[MAX_COMMAND];

    printf("Session 11 - Command History\n");
    printf("-----\n");
    printf("History capacity: %d commands\n", MAX_HISTORY);
    printf("Use 'history' to display commands.\n");
    printf("Use 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }

        if (strcmp(command, "history") == 0)
        {
            show_history();
            continue;
        }

        if (strlen(command) == 0)
            continue;

        add_history(command);
        printf("Command stored in history.\n");
    }

    return 0;
}

```

Output:

```
Shell exiting...
rohita@LaxmiDurga:~$ nano s6.c
rohita@LaxmiDurga:~$ gcc s6.c
rohita@LaxmiDurga:~$ ./a.out
Session 11 - Command History
-----
History capacity: 5 commands
Use 'history' to display commands.
Use 'exit' to quit.

myshell> history

Command History
-----
History is empty.
myshell> exit
Shell exiting...
rohita@LaxmiDurga:~$ _
```

Observation :

- To implement command history using a fixed-size buffer.
- Commands can be stored, retrieved, and displayed, with the oldest command removed when the buffer is full.

Task2: Implement pipeline functionality in a shell. The program should create pipeline structures, store commands, maintain execution order, connect processes, validate the pipeline layout, and test configurations.

Code:

rohita@LaxmiDurga: ~

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    printf("Session 11 - Pipeline Structure\n");
    printf("-----\n");
    printf("Pipeline: ls -l | grep .c\n\n");

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipeline created successfully.\n");
    printf("Execution order:\n");
    printf("1. ls -l\n");
    printf("2. grep .c\n");

    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);
```

```
}  
  
if (pid2 == 0)  
{  
    close(pipefd[1]);  
  
    dup2(pipefd[0], STDIN_FILENO);  
  
    close(pipefd[0]);  
  
    execlp("grep", "grep", ".c", NULL);  
  
    perror("execlp grep");  
    exit(1);  
}  
  
close(pipefd[0]);  
close(pipefd[1]);  
  
waitpid(pid1, NULL, 0);  
waitpid(pid2, NULL, 0);  
  
printf("\nPipeline execution completed successfully.\n");  
  
return 0;  
}
```

Output:

```

rohita@LaxmiDurga: ~
rohita@LaxmiDurga:~$ gcc s61.c
rohita@LaxmiDurga:~$ ./a.out
Session 11 - Pipeline Structure
-----
Pipeline: ls -l | grep .c

Pipeline created successfully.
Execution order:
1. ls -l
2. grep .c
-rw-r--r-- 1 rohita rohita 2736 Sep 18 06:59 92.c
-rw-r--r-- 1 rohita rohita 57 Aug 6 14:47 First.c
-rw-r--r-- 1 rohita rohita 1322 Aug 25 03:28 P-week2.c
-rw-r--r-- 1 rohita rohita 1206 Aug 7 15:18 Practical2.c
-rw-r--r-- 1 rohita rohita 935 Aug 6 15:00 Skill2.c
-rw-r--r-- 1 rohita rohita 654 Aug 7 14:32 Skill3.c
-rw-r--r-- 1 rohita rohita 802 Aug 7 14:48 Skill3Task2.c
-rw-r--r-- 1 rohita rohita 0 Sep 11 09:21 client_to_server
-rw-r--r-- 1 rohita rohita 936 Aug 5 16:22 command_exec.c
-rw-r--r-- 1 rohita rohita 0 Aug 25 04:47 copy.txt
-rw-r--r-- 1 rohita rohita 729 Aug 25 04:43 new.c
-rw-r--r-- 1 rohita rohita 270 Aug 25 04:46 new1.c
-rw-r--r-- 1 rohita rohita 1252 Aug 25 04:55 new12.c
-rw-r--r-- 1 rohita rohita 710 Aug 25 04:42 os-p-2.c
-rw-r--r-- 1 rohita rohita 931 Aug 20 08:27 os.c
-rw-r--r-- 1 rohita rohita 773 Sep 11 09:20 p-6.c
-rw-r--r-- 1 rohita rohita 1542 Sep 10 08:33 p6.c
-rw-r--r-- 1 rohita rohita 657 Sep 11 09:29 p61.c
-rw-r--r-- 1 rohita rohita 689 Sep 12 02:49 p62.c
-rw-r--r-- 1 rohita rohita 926 Sep 15 06:51 p7.c
-rw-r--r-- 1 rohita rohita 853 Sep 15 06:55 p71.c
-rw-r--r-- 1 rohita rohita 1432 Sep 15 08:53 p8.c
-rw-r--r-- 1 rohita rohita 1870 Sep 15 09:05 p81.c
-rw-r--r-- 1 rohita rohita 148 Aug 19 09:50 s1.c
-rw-r--r-- 1 rohita rohita 881 Aug 19 23:47 s12.c
-rw-r--r-- 1 rohita rohita 559 Aug 19 23:57 s2.c
-rw-r--r-- 1 rohita rohita 538 Aug 20 00:00 s21.c
-rw-r--r-- 1 rohita rohita 1661 Sep 18 08:03 s6.c
-rw-r--r-- 1 rohita rohita 1321 Sep 18 08:08 s61.c
-rw-r--r-- 1 rohita rohita 2134 Sep 18 06:46 s9.c
-rw-r--r-- 1 rohita rohita 1657 Sep 18 06:47 s91.c
-rw-r--r-- 1 rohita rohita 1177 Sep 18 06:58 s92.c
-rw-r--r-- 1 rohita rohita 0 Sep 11 09:21 server_to_client
-rw-r--r-- 1 rohita rohita 3735 Aug 22 06:33 sk1.c
-rw-r--r-- 1 rohita rohita 1905 Aug 11 07:08 skil4Task1.c
-rw-r--r-- 1 rohita rohita 968 Aug 7 14:52 skill3Task2.c
-rw-r--r-- 1 rohita rohita 5428 Aug 20 06:36 skillweek2.c
-rw-r--r-- 1 rohita rohita 1287 Aug 20 09:05 skillweek4.c
-rw-r--r-- 1 rohita rohita 4192 Aug 20 06:39 skilweek2.c
-rw-r--r-- 1 rohita rohita 5507 Aug 20 06:58 skilweek21.c
-rw-r--r-- 1 rohita rohita 29 Aug 25 05:00 source.txt
-rw-r--r-- 1 rohita rohita 1885 Aug 20 05:28 sweek2.c
-rw-r--r-- 1 rohita rohita 1855 Aug 20 05:34 sweek2.c.save
-rw-r--r-- 1 rohita rohita 1322 Aug 25 03:29 w-p2.c
-rw-r--r-- 1 rohita rohita 1771 Aug 27 09:42 w5.c
-rw-r--r-- 1 rohita rohita 1305 Aug 27 09:48 w51.c
-rw-r--r-- 1 rohita rohita 3010 Aug 24 14:05 wTask42.c
-rw-r--r-- 1 rohita rohita 1472 Aug 22 06:12 week4.c
-rw-r--r-- 1 rohita rohita 2926 Aug 24 13:41 weweekTask4.c
-rw-r--r-- 1 rohita rohita 920 Aug 27 08:09 week4.c
-rw-r--r-- 1 rohita rohita 811 Aug 27 08:19 week42.c
-rw-r--r-- 1 rohita rohita 2688 Aug 22 06:19 week4Task1.c
-rw-r--r-- 1 rohita rohita 926 Aug 27 08:14 wek41.c

Pipeline execution completed successfully.
rohita@LaxmiDurga:~$

```

Observation: To create a shell pipeline using `pipe()`, `fork()`, `dup2()`, and `exec()`. processes communicate by connecting the output of one process to the input of another and how `waitpid()` maintains the correct execution order.

Session 12

Task 1: Implement a program to create pipes, redirect streams, execute connected commands, close descriptors, synchronize processes, and test data flow.

Code:

rohita@LaxmiDurga: ~

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    printf("Session 12 - Task 1: Pipe Data Flow\n");
    printf("-----\n");
    printf("Command: ls -l | grep .c\n\n");

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        // First process: ls -l
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        // Second process: grep .c
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);
    }
}
```

```

    }

    if (pid2 == 0)
    {
        // Second process: grep .c
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep");
        exit(1);
    }

    // Parent closes unused descriptors
    close(pipefd[0]);
    close(pipefd[1]);

    // Wait for both processes
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("\nData flow completed successfully.\n");

    return 0;
}

```

Output:

```

rohita@LaxmiDurga: ~
rohita@LaxmiDurga:~$ nano s62.c
rohita@LaxmiDurga:~$ gcc s62.c
rohita@LaxmiDurga:~$ ./a.out
Session 12 - Task 1: Pipe Data Flow
-----
Command: ls -l | grep .c
-rw-r--r-- 1 rohita rohita 2736 Sep 18 06:59 92.c
-rw-r--r-- 1 rohita rohita 57 Aug 6 14:47 First.c
-rw-r--r-- 1 rohita rohita 1322 Aug 25 03:28 P-week2.c
-rw-r--r-- 1 rohita rohita 1206 Aug 7 15:18 Practical2.c
-rw-r--r-- 1 rohita rohita 935 Aug 6 15:00 Skill2.c
-rw-r--r-- 1 rohita rohita 654 Aug 7 14:32 Skill3.c
-rw-r--r-- 1 rohita rohita 802 Aug 7 14:48 Skill3Task2.c
prw-r--r-- 1 rohita rohita 0 Sep 11 09:21 client_to_server
-rw-r--r-- 1 rohita rohita 936 Aug 5 16:22 command_exec.c
-rw-r--r-- 1 rohita rohita 0 Aug 25 04:47 copy.txt
-rw-r--r-- 1 rohita rohita 729 Aug 25 04:43 new.c
-rw-r--r-- 1 rohita rohita 270 Aug 25 04:46 new1.c
-rw-r--r-- 1 rohita rohita 1252 Aug 25 04:55 new12.c
-rw-r--r-- 1 rohita rohita 710 Aug 25 04:42 os-p-2.c
-rw-r--r-- 1 rohita rohita 931 Aug 20 08:27 os.c
-rw-r--r-- 1 rohita rohita 773 Sep 11 09:20 p-6.c
-rw-r--r-- 1 rohita rohita 1542 Sep 10 08:33 p6.c
-rw-r--r-- 1 rohita rohita 657 Sep 11 09:29 p61.c
-rw-r--r-- 1 rohita rohita 689 Sep 12 02:49 p62.c
-rw-r--r-- 1 rohita rohita 926 Sep 15 06:51 p7.c
-rw-r--r-- 1 rohita rohita 853 Sep 15 06:55 p71.c
-rw-r--r-- 1 rohita rohita 1432 Sep 15 08:53 p8.c
-rw-r--r-- 1 rohita rohita 1870 Sep 15 09:05 p81.c
-rw-r--r-- 1 rohita rohita 148 Aug 19 09:50 s1.c
-rw-r--r-- 1 rohita rohita 881 Aug 19 23:47 s12.c
-rw-r--r-- 1 rohita rohita 559 Aug 19 23:57 s2.c
-rw-r--r-- 1 rohita rohita 538 Aug 20 00:00 s21.c
-rw-r--r-- 1 rohita rohita 1661 Sep 18 08:03 s6.c
-rw-r--r-- 1 rohita rohita 1321 Sep 18 08:08 s61.c
-rw-r--r-- 1 rohita rohita 1300 Sep 18 08:15 s62.c
-rw-r--r-- 1 rohita rohita 2134 Sep 18 06:46 s9.c
-rw-r--r-- 1 rohita rohita 1657 Sep 18 06:47 s91.c
-rw-r--r-- 1 rohita rohita 1177 Sep 18 06:58 s92.c
prw-r--r-- 1 rohita rohita 0 Sep 11 09:21 server_to_client
-rw-r--r-- 1 rohita rohita 3735 Aug 22 06:33 sk1.c
-rw-r--r-- 1 rohita rohita 1905 Aug 11 07:08 skil4Task1.c
-rw-r--r-- 1 rohita rohita 968 Aug 7 14:52 skill3Task2.c
-rw-r--r-- 1 rohita rohita 5428 Aug 20 06:36 skillweek2.c
-rw-r--r-- 1 rohita rohita 1287 Aug 20 09:05 skillweek4.c
-rw-r--r-- 1 rohita rohita 4192 Aug 20 06:39 skilweek2.c
-rw-r--r-- 1 rohita rohita 5507 Aug 20 06:58 skilweek21.c
-rw-r--r-- 1 rohita rohita 29 Aug 25 05:00 source.txt
-rw-r--r-- 1 rohita rohita 1885 Aug 20 05:28 sweek2.c
-rw-r--r-- 1 rohita rohita 1855 Aug 20 05:34 sweek2.c.save
-rw-r--r-- 1 rohita rohita 1322 Aug 25 03:29 w-p2.c
-rw-r--r-- 1 rohita rohita 1771 Aug 27 09:42 w5.c
-rw-r--r-- 1 rohita rohita 1305 Aug 27 09:48 w51.c
-rw-r--r-- 1 rohita rohita 3010 Aug 24 14:05 wTask42.c
-rw-r--r-- 1 rohita rohita 1472 Aug 22 06:12 weeeek4.c
-rw-r--r-- 1 rohita rohita 2926 Aug 24 13:41 weeeekTask4.c
-rw-r--r-- 1 rohita rohita 920 Aug 27 08:09 week4.c
-rw-r--r-- 1 rohita rohita 811 Aug 27 08:19 week42.c
-rw-r--r-- 1 rohita rohita 2688 Aug 22 06:19 week4Task1.c
-rw-r--r-- 1 rohita rohita 926 Aug 27 08:14 wek41.c
Data flow completed successfully.
rohita@LaxmiDurga:~$

```

Observation:

learned how to create a pipe and connect two processes using pipe(), fork(), and dup2(). understood how the output of one command can be redirected as the input of another command and how unused file descriptors are closed to maintain proper data flow.

Task2:

Code:

Select rohita@LaxmiDurga: ~

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define COMMANDS 3

int main()
{
    int pipes[COMMANDS - 1][2];
    pid_t pids[COMMANDS];

    printf("Session 12 - Task 2: Multiple Pipes\n");
    printf("-----\n");
    printf("Pipeline: ls -l | grep .c | wc -l\n");

    // Create pipes
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return 1;
        }
    }

    // Create processes
    for (int i = 0; i < COMMANDS; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return 1;
        }

        if (pids[i] == 0)
        {
            // First command
            if (i == 0)
            {
                close(pipes[0][0]);
                dup2(pipes[0][1], STDOUT_FILENO);
            }
            // Middle command
            else if (i == 1)
            {
                dup2(pipes[0][0], STDIN_FILENO);
                dup2(pipes[1][1], STDOUT_FILENO);
            }
            // Last command
            else if (i == 2)
            {
                dup2(pipes[1][0], STDIN_FILENO);
            }

            // Close all pipe descriptors
            for (int j = 0; j < COMMANDS - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            // Last command
            else if (i == 2)
            {
                dup2(pipes[1][0], STDIN_FILENO);
            }

            // Execute command
            if (i == 0)
                execlp("ls", "ls", "-l", NULL);
            else if (i == 1)
                execlp("grep", "grep", ".c", NULL);
            else
                execlp("wc", "wc", "-l", NULL);

            perror("execlp");
            exit(1);
        }
    }

    // Parent closes all pipe descriptors
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    // Wait for all children
    for (int i = 0; i < COMMANDS; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    printf("Long pipeline execution completed successfully.\n");
    return 0;
}
```

Output:

```
Data flow completed successfully.
rohita@LaxmiDurga:~$ nano skill6.c
rohita@LaxmiDurga:~$ gcc skill6.c
rohita@LaxmiDurga:~$ ./a.out
Session 12 - Task 2: Multiple Pipes
-----
Pipeline: ls -l | grep .c | wc -l

55
Long pipeline execution completed successfully.
```

Observation : Multiple pipes for handling long command pipelines.To manage file descriptors, coordinate multiple processes, connect input and output streams, and properly clean up resources after execution.